

Two-body low-energy scattering

Theory and implementation: every function, every constant, and why

Pedro Henrique Gesualdo Modesto

São Carlos Institute of Physics — University of São Paulo

August 21, 2026

Abstract

This document takes apart the two-body low-energy scattering code: every function, every term, every parameter, and the reason behind each choice. It is not a user manual — it is the explanation of *why*. Every numerical constant that appears in the code has here the calculation or the measurement that fixed it. The errors made and corrected are collected in Sec. 16, because they are the most instructive part.

Contents

1	What universality means here	3
2	Conventions and units	4
3	The one idea: do not integrate what is already known	4
4	The four potentials	5
4.1	The range R , in closed form	5
4.2	A factor of two in Eq. (121)	6
5	Numerov: the integrator	6
6	The logarithmic grid	7
6.1	The problem	7
6.2	The calculation	7
7	The two starts	8
8	The slope at the edge	9
9	What the code measures: a and r_0	9
9.1	The scattering length	9
9.2	The normalisation, and why it is needed	10
9.3	The effective range	10
10	Counting bound states	10
11	The bound state	11
12	The inverse problem: tuning	11
12.1	Two nested loops	11
13	Every constant, one by one	12

14 The tabulated data	13
14.1 Scale invariance in GUESS	14
14.2 The constant that is tabulated nowhere	14
15 The checks	15
16 Autopsy: the errors made	15
17 Results	16
17.1 Binding energies	16
17.2 Accuracy achieved	17
18 What is missing	17
A The three functions in full	18
A.1 numerov — the integrator	18
A.2 scattering — the three readings	19
A.3 tune — the inverse problem	19
B The data, with values	20

1 What universality means here

At low energy, when the de Broglie wavelength is comparable to or larger than the range of the potential, the detailed shape of the interaction becomes invisible. Only two numbers survive — the **scattering length** a and the **effective range** r_0 — through the effective-range expansion

$$k \cot \delta_0(k) = -\frac{1}{a} + \frac{1}{2} r_0 k^2 + O(k^4). \quad (1)$$

This is what universality means: the same two-parameter description applies across physical domains that have nothing else in common. In the words of Ref. [1], “*the universal behavior in the low-energy limit enables us to draw a parallel between cold atomic gases and nuclear systems.*”

The two systems used throughout this work make the point concrete:

system	a	r_0	E (measured)	domain
^4He dimer	90.4 Å	8.0 Å	−1.62 mK	atomic
deuteron	5.4112 fm	1.7436 fm	−2.224 MeV	nuclear

Nine orders of magnitude apart in energy, five in length, and one is held together by the van der Waals force while the other is held together by the strong interaction. And yet the same two numbers, fed into the same two formulas, reproduce both binding energies.

Shape independence is the mechanism, not the definition

A separate — and often confused — statement is that *different potentials* tuned to the same (a, r_0) give the same low-energy observables. That is not the definition of universality; it is a consequence of the **shape-independent approximation**, Eq. (55) of [1], which says that $\delta_0(k)$ at low energy does not depend on the shape of the potential. As [1] puts it, the fact that four very different potentials can be tuned to the same a and r_0 , “*although remarkable, is directly related to the shape-independent approximation.*”

So the roles are:

universality	two parameters describe low-energy physics across domains
shape independence	why the shape of V beyond those two numbers does not matter
tuning	the numerical procedure that puts a given V at a chosen (a, r_0)

Why: Tuning is a *demonstration technique*, not a physical claim. The tuned parameters are of course different for each potential — a Gaussian and a Lennard-Jones that share (a, r_0) have nothing numerically in common. What they share is everything a low-energy experiment can measure.

When two parameters are not enough

The expansion (1) is a series, and dropping r_0 means keeping only its first term. Whether that is legitimate depends on how small kR is. Reference [1] makes exactly this point for the deuteron: the zero-range formula “*only accounts for $\approx 64\%$ of the deuteron energy*”, because “ *$kR \sim 0.4$ for the deuteron is larger than in the ^4He case.*”

system	$r_0/ a $	zero range	finite range
deuteron	0.32	−36%	−0.05%
^4He dimer	0.089	−8.6%	+0.47%

Why: The deuteron is the textbook example of a shallow bound state, and it is the one that needs the finite-range correction most. Being universal is not a property a system has or lacks in the abstract; it is a statement about how small the expansion parameter happens to be.

2 Conventions and units

The s -wave radial Schrödinger equation, written for $u(r) = r \psi(r)$, is

$$-\frac{\hbar^2}{2m_r} u''(r) + V(r) u(r) = E u(r), \quad (2)$$

with $m_r = m_1 m_2 / (m_1 + m_2)$ the reduced mass. The code sets $\hbar = m_r = 1$ and measures lengths in fm, so

$$\boxed{u'' = 2(V - E) u} \quad (3)$$

which is the only equation the code ever solves, at $E = 0$ and at $E < 0$ alike.

Back to physical units

Comparing (3) with the dimensional form,

$$\frac{2m_r}{\hbar^2} (V_{\text{phys}} - E_{\text{phys}}) = 2(V_{\text{code}} - E_{\text{code}}) \implies E_{\text{phys}} = \frac{\hbar^2}{m_r} E_{\text{code}},$$

and since $\hbar^2/2m_r$ is the constant usually tabulated,

$$\boxed{E_{\text{phys}} = 2 \left(\frac{\hbar^2}{2m_r} \right) E_{\text{code}}} \quad (4)$$

Why: That factor of 2 is easy to lose. The check that would expose it is in Sec. 14: with it wrong, the inferred constant would not come out at 41.47 MeV·fm².

3 The one idea: do not integrate what is already known

Every potential here has a range R . Beyond R we have $V = 0$, and there Eq. (3) has a solution known without computing anything:

$$E = 0 : \quad u'' = 0 \quad \implies \quad u \text{ is a **straight line**,} \quad (5)$$

$$E < 0 : \quad u'' = -2E u \quad \implies \quad u \propto e^{-\kappa r}, \quad \kappa = \sqrt{-2E}. \quad (6)$$

So the code **never integrates past R** . It integrates from the inner boundary out to R , reads the value and the slope there, and glues the analytic form on. All three quantities come out of that gluing:

quantity	comes from
a	where the asymptotic straight line crosses zero
r_0	the integral of the difference between that line and the true solution
E	the energy that makes the solution glue onto the exponential

Why: Integrating beyond R would mean computing numerically something already known exactly, and paying for it in numerical error.

4 The four potentials

The choice is not decorative: each shape stresses a different numerical hypothesis.

Potential	Form	What it stresses
Square well, Eq. (70)	$-v\mu^2$ for $r < 1/\mu$	a discontinuity
Pöschl-Teller, Eq. (116)	$-v\mu^2 / \cosh^2(\mu r)$	smooth and solvable in closed form
Gaussian, Eq. (120)	$-v\mu^2 e^{-\mu^2 r^2}$	tail dying faster than exponentially
Lennard-Jones, Eq. (121)	$\frac{1}{2}(C_{12}/r^{12} - C_6/r^6)$	hard core plus van der Waals tail

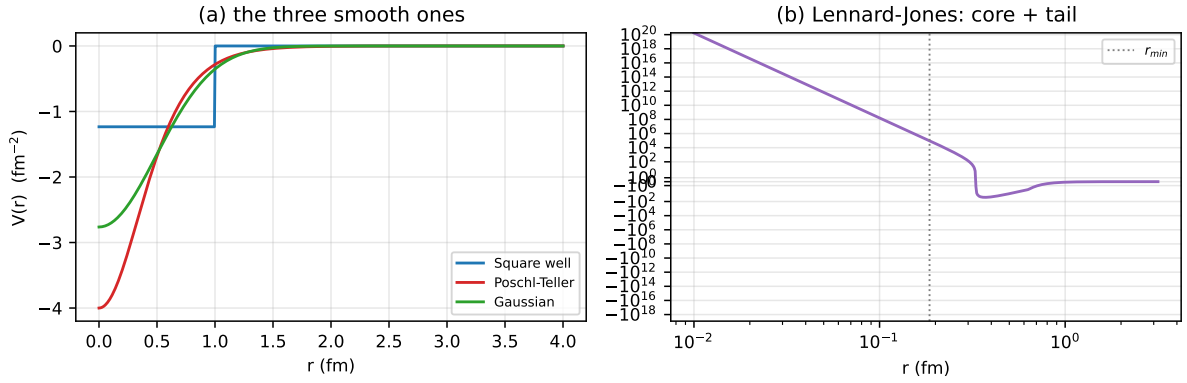


Figure 1: The four potentials, all tuned to unitarity ($|a| \rightarrow \infty$). The Lennard-Jones needs logarithmic axes because its core reaches 10^{10} in code units; the dotted line marks $r_{\min}$, where integration starts.

4.1 The range R , in closed form

Each class needs to know where its potential stops acting. For the well that is exact, $R = 1/\mu$. For the other three the potential never truly vanishes — a Gaussian decays forever — so R is defined as the radius where $|V(R)| = V_{\text{ZERO}}$, and that equation is solved *on paper*:

$$\text{Pöschl-Teller: } \frac{v\mu^2}{\cosh^2(\mu R)} = \varepsilon \quad \Rightarrow \quad R = \frac{1}{\mu} \operatorname{arccosh} \sqrt{\frac{v\mu^2}{\varepsilon}} \quad (7)$$

$$\text{Gaussian: } v\mu^2 e^{-\mu^2 R^2} = \varepsilon \quad \Rightarrow \quad R = \frac{1}{\mu} \sqrt{\ln \frac{v\mu^2}{\varepsilon}} \quad (8)$$

$$\text{Lennard-Jones (tail): } \frac{1}{2} C_6 / R^6 = \varepsilon \quad \Rightarrow \quad R = \left(\frac{C_6}{2\varepsilon} \right)^{1/6} \quad (9)$$

$$\text{Lennard-Jones (core): } \frac{1}{2} C_{12} / r_{\min}^{12} = V_{\text{core}} \quad \Rightarrow \quad r_{\min} = \left(\frac{C_{12}}{2V_{\text{core}}} \right)^{1/12} \quad (10)$$

Why: An earlier version searched for these radii numerically, sweeping a global 40 000-point grid. Solving on paper removed that grid, removed a helper function, and made the result exact instead of limited by the sweep resolution.

4.2 A factor of two in Eq. (121)

Equation (121) of [1] reads

$$V_{LJ}(r) = \frac{\hbar^2}{m_r} \left(\frac{C_{12}}{r^{12}} - \frac{C_6}{r^6} \right),$$

which in the units used here is $C_{12}/r^{12} - C_6/r^6$, with **no** $\frac{1}{2}$. Taken literally, it does not reproduce Table 4 of the same paper. Using the published deuteron parameters $C_{12} = 0.90485319$ and $C_6 = 6.81472$:

	a (fm)	r_0 (fm)
Table 4 of [1]	5.4	1.70
with the $\frac{1}{2}$	5.405	1.699
without it	1.435	1.412

Equations (70) and (120), for the well and the Gaussian, carry the same $\hbar^2/m_r$ prefactor and *do* reproduce Table 3 exactly as printed. The mismatch is therefore specific to Eq. (121); the likely reading is $2m_r$ in its denominator. The code uses the $\frac{1}{2}$.

Why: This is the kind of detail that survives unnoticed because the result *looks* right either way — the curves have the expected shape. Only the numerical comparison against the published table exposes it.

5 Numerov: the integrator

Numerov solves equations of the form $u'' = W(r)u$, which is exactly Eq. (3) with $W = 2(V - E)$. The recursion is

$$u_{i+1} = \frac{(12 - 10f_i)u_i - f_{i-1}u_{i-1}}{f_{i+1}}, \quad f_i = 1 - \frac{h^2 W_i}{12}. \quad (11)$$

Where it comes from

Expanding $u_{i\pm 1}$ in Taylor series and adding,

$$u_{i+1} + u_{i-1} = 2u_i + h^2 u_i'' + \frac{h^4}{12} u_i^{(4)} + O(h^6).$$

A second-order method simply discards the h^4 term. Numerov does not: since $u'' = Wu$, we have $u^{(4)} = (Wu)''$, and that second derivative can be approximated by the *same* central difference, without evaluating anything at new points. Substituting and regrouping gives Eq. (11).

Why: This is why Numerov reaches order h^4 at the cost of a second-order scheme: it never evaluates W away from the grid. The price is a *three-term* recursion — each point depends on the two before it, so an error made at the start travels all the way out.

```
f = 1.0 - h * h * W / 12.0
for i in range(1, POINTS - 1):
    w[i + 1] = ((12.0 - 10.0 * f[i]) * w[i] - f[i - 1] * w[i - 1]) / f[i + 1]
```

6 The logarithmic grid

6.1 The problem

Numerov requires $h\sqrt{|W|} \ll 1$ to represent the solution. Measured across the four potentials on a uniform grid:

potential	h (fm)	$\max W $	$h\sqrt{ W }$
square well	0.00103	8.3×10^{-1}	0.001
Pöschl-Teller	0.00851	2.1×10^0	0.012
Gaussian	0.00401	1.6×10^0	0.005
Lennard-Jones	0.00626	2.0×10^5	2.78

The Lennard-Jones is squeezed from both sides: its core reaches $V = 10^5$, so $\sqrt{2V} = 447$ and the characteristic length there is 0.002 fm, while its $1/r^6$ tail pushes R out to 125 fm. **No uniform step serves both.**

The consequence was measured, and it is a real error: the Lennard-Jones r_0 *drifted* as the number of points doubled, instead of converging.

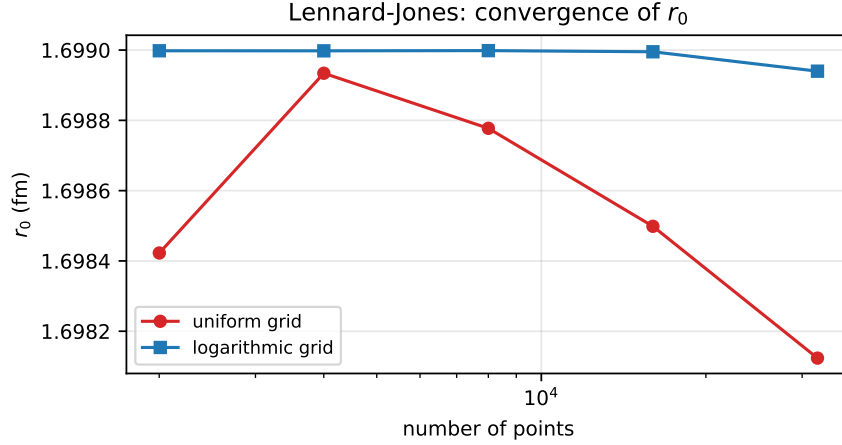


Figure 2: Convergence test. On a uniform grid (red) r_0 never settles. On a logarithmic grid (blue) it locks in at the fifth decimal. A result that moves when the grid is refined is not a result.

6.2 The calculation

With $r = e^x$ and $u = e^{x/2}w(x)$, using $\frac{d}{dr} = e^{-x} \frac{d}{dx}$:

$$u' = e^{-x} \frac{d}{dx} (e^{x/2}w) = e^{-x/2} \left(\frac{1}{2}w + w' \right), \quad (12)$$

$$u'' = e^{-x} \frac{d}{dx} \left[e^{-x/2} \left(\frac{1}{2}w + w' \right) \right] = e^{-3x/2} \left(w'' - \frac{1}{4}w \right). \quad (13)$$

Substituting into Eq. (3), $e^{-3x/2}(w'' - \frac{1}{4}w) = 2(V - E)e^{x/2}w$, so

$$\boxed{w''(x) = \left[2(V(e^x) - E)e^{2x} + \frac{1}{4} \right] w(x)} \quad (14)$$

Two things matter here. First, a $\frac{1}{4}$ **appears** that was not in the original equation — it comes from the substitution $u = e^{x/2}w$, not from the physics. Second, and decisively: Eq. (14) is still of

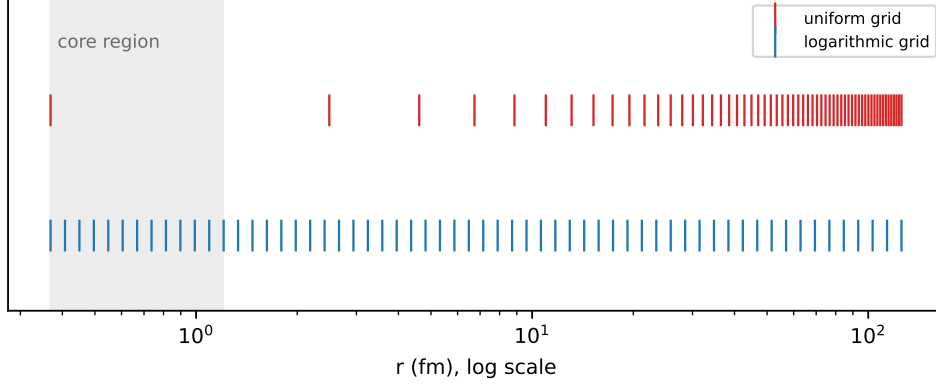


Figure 3: Point distribution, same number of points in both cases. The uniform grid wastes almost everything on the tail and leaves the core with a handful of points; the logarithmic grid does the opposite.

the form $w'' = Ww$, which is the *only* thing Numerov requires. The recursion does not change at all; only what goes into it does.

Why: Changing the independent variable is the standard move when a problem has two very different scales. Here they differ by five orders of magnitude: 0.002 fm in the core against 125 fm in the tail.

```
r_start = pot.r_min if pot.r_min > 0.0 else pot.R * 1e-6
x = np.linspace(math.log(r_start), math.log(pot.R), POINTS)
h = x[1] - x[0]
r = np.exp(x)
W = 2.0 * (pot.V(r) - E) * r * r + 0.25
```

7 The two starts

Numerov needs *two* initial values, and the code picks between two cases:

```
if pot.r_min > 0.0:
    w[1] = h * math.exp(x[0] / 2.0)      # hard core: u = 0, u' = 1 there
else:
    w[0] = math.sqrt(r[0])               # regular origin: u ~ r
    w[1] = math.sqrt(r[1])
```

Regular origin (well, Pöschl-Teller, Gaussian). Near $r = 0$ with finite V , the regular solution behaves as $u \simeq r$. Since $w = u e^{-x/2} = u/\sqrt{r}$, that gives $w \simeq \sqrt{r}$, which is exactly what the code writes.

Hard core (Lennard-Jones). There V blows up and u is exponentially small; we impose $u(r_{\min}) = 0$ with unit slope, and the same conversion $w = u/\sqrt{r}$ gives $w_1 \simeq h e^{x_0/2}$.

Why: Getting the start wrong is not a local mistake: by the three-term recursion, the error propagates through the whole integration. A measurement from this work: for the Lennard-Jones, the fourth-order start correction $u_1 = h + h^3 W_0/6$ amounts to 129% of h . It did not fix that version's problem — the cause was the grid — but it shows the scale of what is at stake.

8 The slope at the edge

Gluing the solution onto the analytic form needs $u(R)$ and $u'(R)$. The value comes out directly; the derivative uses a five-point one-sided difference:

$$\left. \frac{dw}{dx} \right|_R \simeq \frac{25w_N - 48w_{N-1} + 36w_{N-2} - 16w_{N-3} + 3w_{N-4}}{12h} + O(h^4), \quad (15)$$

and the conversion back to physical radius comes from the first line of the calculation in Sec. 6:

$$\left. \frac{du}{dr} \right|_R = e^{-x_N/2} \left(\frac{1}{2}w_N + \left. \frac{dw}{dx} \right|_R \right). \quad (16)$$

Why: Two reasons for the one-sided formula instead of a central difference. It is order h^4 , matching the rest of the method — a central one would be $O(h^2)$ and would spoil the global order. And it uses *interior points only*, so it never crosses the square well's discontinuity.

The discontinuity finding

If a Numerov step straddles the well's jump, the method assumes W is smooth across the three points it uses — and it is not. The convergence order was measured:

n	error in a	measured order
2001	8.9×10^{-4}	—
4001	4.5×10^{-4}	1.00
8001	2.2×10^{-4}	1.00
16001	1.1×10^{-4}	1.00

Order exactly 1, not 4. With the grid ending on the edge and the one-sided derivative, the same case drops to 1.9×10^{-10} — **six orders of magnitude**, with the same number of points.

9 What the code measures: a and r_0

9.1 The scattering length

Beyond R the solution is the straight line $u = C(1 - r/a)$. It crosses zero at $r = a$, and knowing $u(R)$ and $u'(R)$ fixes the line:

$$a = R - \frac{u(R)}{u'(R)} \quad (17)$$

Why: This is an *exact reading*, not a fit. No least squares, no extrapolation. The only source of error is the integration out to R .

The code handles $u'(R) = 0$ separately: the line is horizontal, never crosses zero, and $a = \infty$. That is exact unitarity, not a failure.

```
if slope == 0.0:
    a = math.inf
else:
    a = pot.R - u[-1] / slope
```

9.2 The normalisation, and why it is needed

Equation (3) is homogeneous: if u is a solution, so is Cu . The start chose an arbitrary normalisation, so u is not yet comparable with anything. We fix it by requiring that it meet the line at the edge:

$$u \longleftarrow u \frac{1 - R/a}{u(R)} \implies u(R) = g_0(R), \quad g_0(r) \equiv 1 - \frac{r}{a}. \quad (18)$$

Why: Without this the effective-range integral would not be defined — it compares u against g_0 , and the comparison only makes sense once both are on the same scale.

9.3 The effective range

$$r_0 = 2 \int_0^\infty [g_0(r)^2 - u(r)^2] dr. \quad (19)$$

The integrand measures *how far the true solution departs from the line*, which is why it is a measure of the size of the potential. Outside the range $u = g_0$ and the integrand vanishes identically, so the integral does end at R .

Two details in the code:

The factor r . On the logarithmic grid $dr = r dx$, so the integral becomes $\int(\dots) r dx$. That is where the `* r` in the Simpson call comes from.

The analytic piece. The grid starts at r_{start} , not at zero. The stretch $[0, r_{\text{start}}]$ is added by hand: there u is either zero (inside a hard core) or proportional to r , and in both cases its contribution is $O(r_{\text{start}}^3)$ while the line's is $O(r_{\text{start}})$, leaving

$$2 \int_0^s g_0^2 dr = 2 \left(s - \frac{s^2}{a} + \frac{s^3}{3a^2} \right), \quad s \equiv r_{\text{start}}. \quad (20)$$

Why: This term used to apply only to the Lennard-Jones. Measurement showed the square well carried a residual error of 2.4×10^{-6} in r_0 that was exactly this missing piece. Making the term unconditional removed an `if` and improved precision by three orders of magnitude — down to 5.6×10^{-12} .

```
r0 = 2.0 * simpson((line ** 2 - u ** 2) * r, x=x)
s = r_start
r0 = r0 + 2.0 * (s - s ** 2 / a + s ** 3 / (3.0 * a ** 2))
```

10 Counting bound states

The code counts *zeros of the zero-energy solution*. It is a striking fact that the scattering solution, at $E = 0$, knows how many bound states the potential has.

Reference [1] treats this as a mandatory check rather than as a result: “*checking the number of nodes of the radial function is necessary to guarantee that it is indeed the situation we wanted to reproduce*”, expecting a nodeless $u(r)$ for $a < 0$ and one node for $a > 0$.

There is a subtlety. Beyond the edge the solution is the line $g_0 = 1 - r/a$, which crosses zero at $r = a$. If $a > R$ that zero is physical and counts. If a is effectively infinite, the zero sits at infinity and does not — hence the test `R < a < 1e4`.

```
nodes = 0
for i in range(1, POINTS - 1):
    if u[i] * u[i + 1] < 0.0:
        nodes = nodes + 1
if pot.R < a < 1e4:
    nodes = nodes + 1
```

Why: The count is not optional. Two solutions can share (a, r_0) and differ in node count — and then they are *not* the same physical state. Every tuning ends by checking it.

A bug that lived here: the loop starts at $i = 1$, not $i = 0$. Since $u_0 = 0$ by construction, including the first point counted a spurious sign change and inflated the count by one for *every* potential.

11 The bound state

Beyond the edge, with $E < 0$, the solution must be the decaying exponential. That gives the matching condition

$$g(E) \equiv u'(R) + \kappa u(R) = 0, \quad \kappa = \sqrt{-2E} \quad (21)$$

Why: We write the *sum*, not $u'/u + \kappa$, because $u(R)$ can pass through zero — the deuteron has a node — and the division would invent a spurious pole for the root finder to chase. This cost hours: with the ratio, the helium dimer came out completely wrong.

Why there is no search

The finite-range formula already delivers the answer to about 1%. It is the guess, and `brentq` merely refines it within $[1.8g, 0.3g]$:

```
return brentq(gap, 1.8 * guess, 0.3 * guess, xtol=1e-15)
```

Why: An earlier version scanned 120 energies on a logarithmic grid just to find where the function changed sign. That was waste: the approximate answer was already in hand and was being ignored. Replacing the scan by the guess made the calculation $3.5\times$ faster *and* more accurate. There is also a conceptual bonus: the bracket always working is itself a statement that the effective-range expansion is good.

12 The inverse problem: tuning

Given a target (a, r_0) , find the parameters. It is a nonlinear 2×2 system, and a has *poles*: it jumps from $+\infty$ to $-\infty$ every time a new bound state appears.

12.1 Two nested loops

<code>tune</code>	outer loop: moves the scale until r_0 matches
<code>match_a</code>	inner loop: moves the strength until $1/a$ matches
<code>find_root</code>	widens a bracket until the sign flips, then bisection

Each loop solves *one* equation in *one* variable. Inside the outer loop, `r0_error` runs the entire inner loop again for every scale it tries — which is why this is the slow part, about 35 of the 54 seconds.

The decision that makes it work: chase $1/a$

Why: a has poles; $1/a$ is smooth and crosses zero. Bisection *dies* on a pole and thrives on a zero. As a bonus, unitarity becomes $1/a = 0$ and needs no special case at all.

```
if math.isinf(a_target):
    inv_a_target = 0.0
else:
    inv_a_target = 1.0 / a_target
```

Why not a 2D Newton

The level sets of $1/a$ and r_0 in the (strength, scale) plane are *nearly parallel* over much of the domain. A Newton solver on both equations at once would meet an ill-conditioned Jacobian exactly there, and take large steps in the wrong direction. Nested bisection cannot diverge once the sign is bracketed — slower per iteration and far more reliable for code someone else will run.

The six return values

`tune` returns (strength, scale, a, r0, nodes, ok). The last is a flag: false if r_0 failed to converge *or* if the node count missed its target. Without it, a tuning that landed on the wrong state would pass unnoticed.

Why: What comes out of `tune` is a different pair of numbers for every potential — there is no shared parameter. What is shared is the pair (a, r_0) , and with it everything a low-energy measurement can see. That is the point of Sec. 1.

13 Every constant, one by one

No number in the code is a guess. This table is the justification for each.

constant	value	where	justification
V_ZERO	10^{-12}	defines R	Below this the potential is treated as gone. Measured: cutting at 10^{-8} truncates the Lennard-Jones tail and misses r_0 by 0.16%; 10^{-12} closes to 0.004%.
V_CORE	10^5	LJ start	Where integration begins inside the core. Ref. [1] prescribes $V(r_{\min}) \sim 10^{10}$; we measured that 10^3 breaks the physics (r_0 off by 0.2%) while anything from 10^4 up changes nothing. Starting deeper only costs stiffness.
POINTS	8001	grid size	Total error is truncation (falls with n) plus roundoff (grows with n); 8001 sits at the minimum. The three smooth potentials are at 10^{-11} and do not care. The Lennard-Jones sets the value: past ~ 8000 its r_0 <i>degrades</i> again — 3×10^{-5} at 32001, 3×10^{-4} at 64001.
r_start	$R \times 10^{-6}$	regular origin	Where the logarithmic grid begins when there is no core. It cannot be 0 because $\ln 0$ does not exist. The stretch below it enters analytically.
[1.8g, 0.3g]	—	brentq	Bracket around the finite-range guess. It works on both systems tested; that it works at all is a statement about the quality of the expansion.
1.3	—	find_root	Bracket widening factor, up to 40 attempts. If no sign change is found it raises rather than returning garbage.
$10^{-7}, 10^{-6}$	—	tolerances	Convergence of r_0 in tune and in the tests. Measured: the tuning hits its target to 10^{-10} , comfortably inside them.

14 The tabulated data

Five dictionaries, all traceable:

TARGETS	Table 2 of [1]: the three targets. The bound-state count is <i>not</i> a column of that table; it comes from the text of Sec. 4.5.
PUBLISHED	Tables 3 and 4: the published parameters. Used as the initial guess <i>and</i> as the validation target.
SYSTEMS	Table 1: two real systems, with measured energies.
GUESS	The deuteron solution, used as a starting point for other systems.
SOURCES	The three references with DOI, plus the D1 divergence.

14.1 Scale invariance in GUESS

The problem is scale invariant: multiplying all lengths by L takes $(a, r_0) \rightarrow (La, Lr_0)$. So attacking another system only needs the guess rescaled:

$$\mu \rightarrow \frac{\mu}{L}, \quad C_{12} \rightarrow C_{12} L^6. \quad (22)$$

The L^6 is because C_{12} multiplies r^{-12} .

14.2 The constant that is tabulated nowhere

$\hbar^2/2m_r$ appears in no table. We obtain it by inverting the zero-range formula on the published pair (a, E_{zr}) :

$$E_{zr} = -\frac{\hbar^2/2m_r}{a^2} \implies \frac{\hbar^2}{2m_r} = -E_{zr} a^2. \quad (23)$$

system	inferred	literature	deviation
deuteron	41.462 MeV·fm ²	41.47	0.02%
⁴ He dimer	12.095 K·Å ²	12.12	0.21%

Why: This is a **test**, not a convenience. Recovering both known constants proves Table 1 is internally consistent — and, incidentally, that the factor of 2 in Eq. (4) is right.

The three tunings that do not match, and why

Nine of the twelve tunings land within 0.2% of the published parameters. Three do not, and they do not fail for the same reason.

Two of them are the D1 divergence. (nn, Lennard-Jones) at 2.68% and (deuteron, Pöschl-Teller) at 2.65%. The cause is *not* numerical: refining the grid moves the deviation from 2.70% to 2.69%. Table 3 of [1] reports $r_0 = 1.73$ for the mPT deuteron row and Table 4 reports $r_0 = 2.71$ for the Lennard-Jones nn row, while Table 2 of the same paper lists the targets as 1.70 and 2.70. Feeding the published parameters back through the solver confirms it: they produce $r_0 = 1.7300$ and 2.7080, not the targets. We tune to the target and land elsewhere — correctly.

The third is a flat direction, not a disagreement. (deuteron, Lennard-Jones) sits 1.13% away, but here the published parameters *do* reproduce the target: they give $r_0 = 1.6990$ against 1.7000, an error of 0.06%. Both parameter pairs are therefore valid. What happens is that the two Lennard-Jones coefficients are strongly correlated. Starting from the published pair and moving to ours:

change	effect on r_0
C_{12} alone, +0.47%	−1.09%
C_6 alone, +1.12%	+1.18%
both together	+ 0.05%

Each coefficient on its own moves r_0 by about 1%; together they cancel. The inversion $(a, r_0) \rightarrow (C_{12}, C_6)$ has a nearly flat direction, so the observable is pinned down far more tightly than the parameters that produce it. This is worth stating plainly because the naive reading — “the parameters disagree by 1%, so something is wrong” — is exactly backwards. Note also that Lennard-Jones is the *most* sensitive of the four potentials ($d \ln r_0 / d \ln C_{12} = 2.28$, against 0.42 for the well), so the flatness comes from the correlation between the two coefficients, not from any insensitivity of the potential.

General rule: a discrepancy that survives refinement is never numerical.

15 The checks

`test_lab.py` runs eight checks, and every one compares against something the code did **not** compute:

check	external anchor
a of the well	Eq. (80) of [1], closed form
r_0 of the well	Eq. (92) of [1], closed form
$r_0/R = 1$ at the poles	exact prediction, caption of Fig. 6 of [1]
bound state	analytic condition $k \cot(kR) = -\kappa$
node counts	Table 2, all 12 cases
tuning	Tables 3 and 4, all 12 cases
inferred constants	41.47 and 12.12 from the literature
grid convergence	halving the points may change nothing

Why: No check says “same as last time”. Such a check would only prove the code is consistent with its own mistakes.

The last one is what *found* the uniform-grid bug. It halves the points rather than doubling them, deliberately: since the total error has a minimum, the useful question is “have we converged coming from below?” and not “what happens deep in the roundoff regime?”.

16 Autopsy: the errors made

This section exists because the cause of an error is usually worth more than the result.

1. Numerov dropping from order 4 to order 1

Symptom: the square well converged far too slowly. **Cause:** a step straddled the discontinuity; Numerov assumes W is smooth across the three points it uses. **Fix:** the grid ends exactly on the edge and the derivative uses interior points only. **Gain:** six orders of magnitude.

2. The Lennard-Jones r_0 drifting

Symptom: $1.74257 \rightarrow 1.74215 \rightarrow 1.74165$ as the points doubled. **Diagnosis:** two wrong hypotheses were discarded before the right one — it was not the tail (truncating at 30 fm kept the drift) and not the start (a fourth-order correction changed nothing). It was $h\sqrt{|W|} = 2.78$ in the core. **Fix:** logarithmic grid. **Confirmation:** the new value agrees with an independent method to seven digits.

3. Matching inside the well

Symptom: the well’s energy came out 0.44% wrong. **Cause:** the match used the *last* point where the potential still acted — which, in a well, is inside, where $V \neq 0$. **Fix:** match outside.

4. The spurious node at the origin

Symptom: every potential reported one extra bound state. **Cause:** $u_0 = 0$ by construction counted as a sign change. **Fix:** start the loop at $i = 1$.

5. The inner term that only applied to the Lennard-Jones

Symptom: residual error of 2.4×10^{-6} in the well's r_0 . **Cause:** the logarithmic grid does not cover $[0, r_{\text{start}}]$, and the analytic term sat inside an **if**. **Fix:** make it unconditional — one **if** fewer and three orders of magnitude better.

6. Citations taken on trust

Symptom: three potentials were attributed to the wrong equation numbers (74 instead of 70, 118 instead of 120, 120 instead of 121), and the node counting was attributed to a theorem that the paper never mentions. **Cause:** the citations were written from memory rather than checked against the source. **Fix:** a full pass with the paper open, which also turned up the factor of two in Sec. 4.2. **Lesson:** a plausible citation and a verified citation look identical on the page.

17 Results

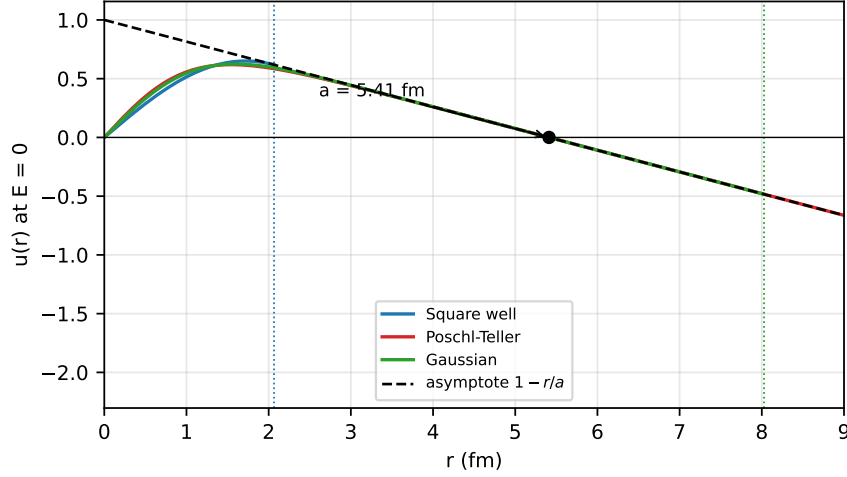


Figure 4: Three potentials tuned to the same (a, r_0) of the deuteron: the wavefunctions are *identical outside* the range and *completely different inside*. Dotted lines mark where each potential ends. Everything a low-energy experiment can see lives outside.

17.1 Binding energies

method	deuteron ($r_0/ a = 0.32$)		^4He dimer ($r_0/ a = 0.089$)	
	E (MeV)	deviation	E (K)	deviation
experiment	-2.224	—	-1.620×10^{-3}	—
zero range	-1.416	-36.3%	-1.480×10^{-3}	-8.6%
finite range	-2.223	-0.05%	-1.628×10^{-3}	+0.47%
square well	-2.204	-0.91%	-1.627×10^{-3}	+0.46%
Pöschl-Teller	-2.227	+0.11%	-1.628×10^{-3}	+0.47%
Gaussian	-2.214	-0.47%	-1.627×10^{-3}	+0.46%
Lennard-Jones	-2.203	-0.92%	—	—

Two systems separated by nine orders of magnitude in energy, described by the same two numbers and the same two formulas. That is the universality of Sec. 1, measured.

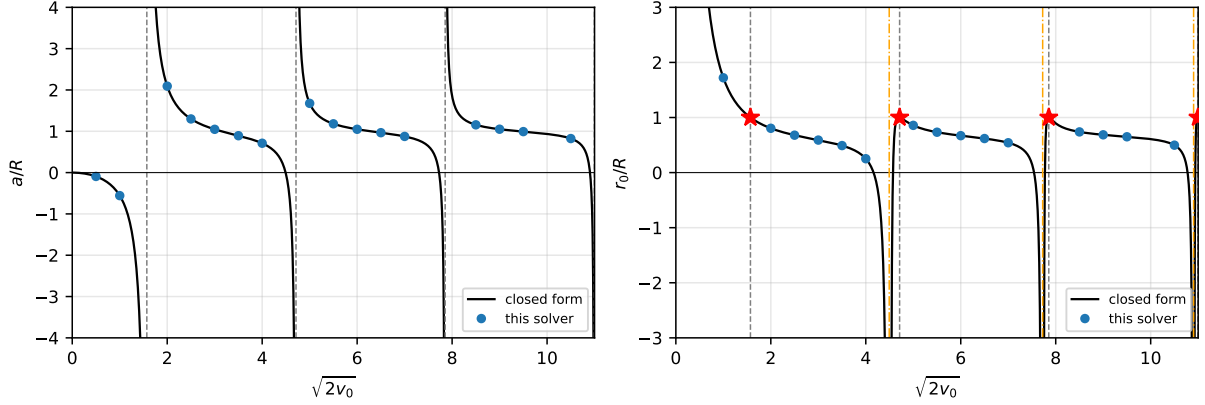


Figure 5: Reproduction of Figs. 5 and 6 of [1]. Left: a/R diverges at $\sqrt{2}v_0 = \pi/2 + n\pi$, where a new bound state appears. Right: at each of those points $r_0/R = 1$ *exactly* (red stars), as stated in the caption of Fig. 6 of [1]. Orange lines mark the *other* family of singularities, the roots of $\tan x = x$, where $a = 0$ and it is r_0 that diverges.

The residual spread of about 1% *among* the potentials is the shape dependence — the $O(k^4)$ terms, the only thing still distinguishing a square well from a Lennard-Jones once a and r_0 are fixed.

17.2 Accuracy achieved

check	error
a of the well against Eq. (80)	1.9×10^{-10}
r_0 of the well against Eq. (92)	5.6×10^{-12}
bound state against $k \cot(kR) = -\kappa$	3.4×10^{-10}
r_0/R at the four poles	1.0000000000
solver against closed forms (16 points)	8.4×10^{-9}
tuning hitting its target	10^{-10}

18 What is missing

No error bars. No number here carries an estimated discretisation uncertainty.

No Aziz HFD-B potential. Reference [1] also benchmarks against the realistic *ab initio* He–He potential. This work uses four model potentials only.

No variable phase method. An independent second route to a , which would give a cross-check on the least reliable number here.

No finite- k phase shift. Everything is done at $E = 0$, with r_0 from the integral. Computing $\delta_0(k)$ and fitting Eq. (1) would be an independent validation — and is the test that would settle the Lennard-Jones r_0 , reliable only to $\sim 10^{-6}$ and limited by roundoff.

s-wave only, valid while $kR \ll 1$.

References

- [1] M. Macêdo-Lima and L. Madeira, *Scattering length and effective range of microscopic two-body potentials*, Rev. Bras. Ensino Fís. **45**, e20230079 (2023). doi:10.1590/1806-9126-RBEF-2023-0079

- [2] R. W. Hackenburg, *Low-energy neutron-proton effective range parameters*, Phys. Rev. C **73**, 044002 (2006). doi:10.1103/PhysRevC.73.044002
- [3] W. Cencek *et al.*, *Effects of adiabatic, relativistic, and QED corrections on the pair potential of helium*, J. Chem. Phys. **136**, 224303 (2012). doi:10.1063/1.4712218

A The three functions in full

The body of the central functions, so that no line is left unexplained. The numbered comments refer to the notes below each block.

A.1 numerov — the integrator

```
def numerov(pot, E):
    r_start = pot.r_min if pot.r_min > 0.0 else pot.R * 1e-6 # (1)

    x = np.linspace(math.log(r_start), math.log(pot.R), POINTS) # (2)
    h = x[1] - x[0]
    r = np.exp(x)
    W = 2.0 * (pot.V(r) - E) * r * r + 0.25 # (3)
    f = 1.0 - h * h * W / 12.0 # (4)

    w = np.zeros(POINTS)
    if pot.r_min > 0.0:
        w[1] = h * math.exp(x[0] / 2.0) # (5)
    else:
        w[0] = math.sqrt(r[0]) # (6)
        w[1] = math.sqrt(r[1])
    for i in range(1, POINTS - 1):
        w[i + 1] = ((12.0 - 10.0*f[i]) * w[i] - f[i-1]*w[i-1]) / f[i+1] # (7)

    u = np.exp(x / 2.0) * w # (8)

    dwdx = (25.0*w[-1] - 48.0*w[-2] + 36.0*w[-3]
            - 16.0*w[-4] + 3.0*w[-5]) / (12.0 * h) # (9)
    slope = math.exp(-x[-1] / 2.0) * (0.5 * w[-1] + dwdx) # (10)

    return r, u, x, h, slope, r_start
```

1. Where to start: outside the core if there is one, otherwise near the origin, because $\ln 0$ does not exist. The skipped stretch enters analytically.
2. The grid is uniform in $x = \ln r$ and ends *exactly* at $\ln R$.
3. The effective potential of Eq. (14). The r^2 comes from e^{2x} ; the 0.25 is the $\frac{1}{4}$ the substitution generated.
4. Numerov's f . Note it is a *vector*, computed once for the whole grid before the loop.
5. Hard-core start: $u = 0$, $u' = 1$, converted through $w = u/\sqrt{r}$.
6. Regular start: $u \simeq r$, hence $w \simeq \sqrt{r}$. Two points are needed because the recursion has three terms.
7. The recursion. It is the only line that actually integrates the equation.
8. Back from w to u . From here on everything is in physical units.
9. Five-point one-sided derivative, order h^4 , interior points only.
10. Conversion of the derivative to radius: $du/dr = e^{-x/2}(w/2 + dw/dx)$.

A.2 scattering — the three readings

```
def scattering(pot):
    r, u, x, h, slope, r_start = numerov(pot, 0.0) # (1)

    if slope == 0.0:
        a = math.inf # (2)
    else:
        a = pot.R - u[-1] / slope

    u = u * (1.0 - pot.R / a) / u[-1] # (3)
    line = 1.0 - r / a

    r0 = 2.0 * simpson((line ** 2 - u ** 2) * r, x=x) # (4)
    s = r_start
    r0 = r0 + 2.0 * (s - s**2 / a + s**3 / (3.0 * a**2)) # (5)

    nodes = 0
    for i in range(1, POINTS - 1): # (6)
        if u[i] * u[i + 1] < 0.0:
            nodes = nodes + 1
    if pot.R < a < 1e4: # (7)
        nodes = nodes + 1

    return a, r0, nodes
```

1. Zero energy: the only case in which the exterior solution is a straight line.
2. A horizontal line means exact unitarity, and $a = \infty$ is the right answer — not an error to be avoided.
3. The normalisation. Without it the following integral would not be defined, because the equation is homogeneous.
4. The integral of Eq. (19). The $* r$ is the Jacobian $dr = r dx$.
5. The stretch $[0, r_{\text{start}}]$, done on paper.
6. The loop starts at 1: including $u_0 = 0$ would count a spurious node.
7. The exterior node at $r = a$, which exists only if a is finite and larger than R .

A.3 tune — the inverse problem

```
def tune(name, a_target, r0_target, strength, scale, nodes_target=None):
    if math.isinf(a_target):
        inv_a_target = 0.0 # (1)
    else:
        inv_a_target = 1.0 / a_target

    for step in range(25): # (2)
        strength = match_a(name, scale, strength, inv_a_target) # (3)
        a, r0, nodes = scattering(POTENTIALS[name](strength, scale))
        if abs(r0 - r0_target) < 1e-7:
            break

    def r0_error(trial_scale): # (4)
        s = match_a(name, trial_scale, strength, inv_a_target)
        a2, r02, n2 = scattering(POTENTIALS[name](s, trial_scale))
        return r02 - r0_target

    scale = find_root(r0_error, scale)
```

```

strength = match_a(name, scale, strength, inv_a_target) # (5)
a, r0, nodes = scattering(POTENTIALS[name](strength, scale))

ok = abs(r0 - r0_target) < 1e-6
if nodes_target is not None and nodes != nodes_target:
    ok = False # (6)

return strength, scale, a, r0, nodes, ok

```

1. Unitarity with no special case: $a = \infty$ is simply $1/a = 0$.
2. Twenty-five attempts at most. Leaving early means it converged.
3. Inner loop: fixes the strength so that $1/a$ matches, scale held.
4. This nested function is the cost of the algorithm: for every scale it tries, it runs the *entire* inner loop again.
5. One last adjustment of the strength, because the scale moved last.
6. The `ok` flag. Without it, a tuning that landed on the wrong state would pass unnoticed.

B The data, with values

TARGETS — Table 2 of [1]

case	a (fm)	r_0 (fm)	bound states
nn (neutron-neutron)	-18.5	2.7	0
unitarity	$\pm\infty$	1.0	0
deuteron	5.4	1.7	1

The bound-state column is not printed in Table 2; it comes from the text of Sec. 4.5 of [1], which asks for a nodeless $u(r)$ when $a < 0$ and one node when $a > 0$.

PUBLISHED — Tables 3 and 4, against what the tuning returns

case	potential	p_1 obtained	p_1 published	p_2 obtained	p_2 published	deviation
nn	well	1.109228	1.1096	0.391087	0.3918	0.18%
nn	Pöschl-Teller	0.907086	0.9071	0.799622	0.7991	0.07%
nn	Gaussian	1.211903	1.2121	0.567906	0.5672	0.12%
nn	Lennard-Jones	9.760657	9.8667	3.005560	3.0884	2.68%
unit.	well	1.233701	1.2337	1.000000	1.0000	0.00%
unit.	Pöschl-Teller	1.000000	1.0000	2.000000	2.0000	0.00%
unit.	Gaussian	1.342002	1.3420	1.435224	1.4349	0.02%
unit.	Lennard-Jones	0.264625	0.2646	0.000341	0.00034	0.00%
deut.	well	1.758644	1.7575	0.499215	0.5000	0.16%
deut.	Pöschl-Teller	1.423895	1.4388	0.886001	0.8631	2.65%
deut.	Gaussian	1.910904	1.9102	0.674817	0.6754	0.09%
deut.	Lennard-Jones	6.846862	6.8147	0.915045	0.9049	1.13%

The two rows in bold are the D1 divergence: the published parameters achieve $r_0 = 2.71$ and 1.73 , while Table 2 of the same paper asks for 2.70 and 1.70 . Refining the grid does not move these deviations. The third row above 0.2%, (deut., Lennard-Jones) at 1.13%, is not of that kind: there the published parameters do reach the target, and the gap is the flat direction in (C_{12}, C_6) discussed above.

SYSTEMS — **Table 1, the two real systems**

system	a	r_0	measured E	inferred $\hbar^2/2m_r$
deuteron	5.4112 fm	1.7436 fm	−2.224 MeV	41.462 MeV·fm ²
⁴ He dimer	90.4 Å	8.0 Å	−1.62 mK	12.095 K·Å ²